Notes on Time Series and Trends

LOESS (Locally Estimated Scatterplot Smoothing)

Local regression (https://en.wikipedia.org/wiki/Local_regression)

Local regression or **local polynomial regression**,[1] also known as **moving regression**,[2] is a generalization of the moving average and polynomial regression.[3] Its most common methods, initially developed for scatterplot smoothing, are **LOESS**(**locally estimated scatterplot smoothing**) and **LOWESS** (**locally weighted scatterplot smoothing**), both pronounced /ˈloʊɛs/ *LOH-ess*. They are two strongly related non-parametric regression methods that combine multiple regression models in a *k*-nearest-neighbor-based meta-model. In some fields, LOESS is known and commonly referred to as Savitzky–Golay filter[4][5] (proposed 15 years before LOESS).

LOESS and LOWESS thus build on "classical" methods, such as linear and nonlinear least squares regression. They address situations in which the classical procedures do not perform well or cannot be effectively applied without undue labor. LOESS combines much of the simplicity of linear least squares regression with the flexibility of nonlinear regression. It does this by fitting simple models to localized subsets of the data to build up a function that describes the deterministic part of the variation in the data, point by point. In fact, one of the chief attractions of this method is that the data analyst is not required to specify a global function of any form to fit a model to the data, only to fit segments of the data.

The trade-off for these features is increased computation.

LOESS (Locally Estimated Scatterplot Smoothing), sometimes called LOWESS, is a non-parametric method for fitting a smooth curve to scatterplot data.

Instead of trying to find a single mathematical function (like a straight line or a complex polynomial) that fits the *entire dataset* at once, LOESS fits multiple, simple models to *localized subsets* of the data point by point.

Here is the exact step-by-step process of how the algorithm generates the smooth curve:

1.Select a target point:

The algorithm picks a specific x-value along the axis where it wants to calculate the smoothed y -value.

2.Define the local neighborhood:

It identifies a "window" of nearby data points centered around that x-value. The size of this window is determined by a parameter called the **span** or bandwidth (often denoted as α). For example, if the span is 0.3, the algorithm looks at the 30% of the dataset closest to the target x -value.

3.Assign weights based on distance:

Points inside the neighborhood are assigned a weight based on how close they are to the target x-value. The closest points get the highest weights and have the most influence on the fit. Points further away get lower weights, and points outside the neighborhood get a weight of zero. This is typically done using a **tricube weight function**.

4.Fit a local regression:

Using only the points within the neighborhood and their assigned weights, the algorithm calculates a **weighted least squares regression**. This is usually a simple linear (degree 1) or quadratic (degree 2) polynomial fit.

5.Record the smoothed value:

The algorithm looks at where this locally fitted line crosses the target x-value and records that specific y-value as the smoothed point. The rest of the local line is discarded.

6.Repeat for all points:

The algorithm moves to the next x-value on the scatterplot and repeats the entire process. By connecting all these individually estimated y-values together, it forms the final, continuous smooth curve.

Key Parameters that Control LOESS

1. **Span (α):** This is the most critical parameter. A small span uses a very small neighborhood, resulting in a "wiggly" curve that captures a lot of local noise. A large span uses a wide neighborhood, creating a much smoother, stiffer curve that might miss actual local trends.

2. **Degree of Polynomial (λ):** A degree of 1 fits a straight line in each local window (often called LOWESS). A degree of 2 fits a small parabola (LOESS), which tends to capture peaks and valleys in the data more accurately.

Pros and Cons:

The main advantage of LOESS is its flexibility; you don't need to specify a global mathematical equation beforehand, making it excellent for exploratory data analysis and visualizing complex, non-linear relationships. The primary drawback is that it is computationally intensive (since it runs a regression for every single point) and it cannot reliably extrapolate data outside the range of your existing x-values.

Spline Smoothing

Spline smoothing works by dividing your data into separate segments and fitting a simple polynomial curve—typically a cubic equation—within each segment. These individual curves are then mathematically welded together at boundaries called **knots** so that the overall curve is completely continuous and the transitions are invisible.

The Core Mechanism

To build a smoothing spline, the algorithm balances two competing goals: fitting the data points closely, and keeping the resulting curve from becoming too jagged. It achieves this through three main structural rules:

- **Piecewise Polynomials:** Instead of forcing one massive, highly complex equation to fit all the data, the algorithm uses low-degree polynomials (like $y = ax^3 + bx^2 + cx + d$) for localized stretches of data along the x-axis.

- **Continuity Constraints:** To ensure the final curve doesn't have sharp corners or sudden bends at the knots where two segments meet, the algorithm strictly enforces that adjacent curves must share the exact same position, slope (first derivative), and curvature (second derivative) at the boundary.

- **The Roughness Penalty:** A true *smoothing spline* actually places a knot at every single unique x-value in your dataset. To prevent the curve from violently zig-zagging to perfectly connect every dot (overfitting), it minimizes a specific loss function:

$$Loss = SumofSquaredErrors + \lambda \int [f''(x)]^2 dx$$

- The first half pulls the curve toward the data points. The second half (the integral of the squared second derivative) acts as a mathematical penalty for "wiggliness" or extreme curvature.

Controlling the Fit

The entire shape of a smoothing spline is dictated by λ, the smoothing parameter:

- **When λ is near 0:** The penalty for wiggliness is practically ignored. The spline will aggressively bend to pass through every data point, resulting in a highly overfit, noisy curve.
- **When λ is extremely large:** The curvature penalty dominates the equation. The curve is forced to have essentially zero curvature, effectively turning the spline into a standard straight-line linear regression.
- **Finding the balance:** Statistical software typically finds the optimal, balanced value for λ automatically using a technique called Generalized Cross-Validation (GCV).

Splines vs. LOESS

While LOESS uses a sliding window to run a brand-new regression calculation for every single point you want to predict, a spline model estimates a single, globally defined set of mathematical coefficients upfront. Because of this, once a spline model is fitted, calculating new predictions is computationally instantaneous. Splines are also the mathematical foundation for more advanced architectures, like Generalized Additive Models (GAMs).

Why Spline Smoothing is More Efficient than LOESS

To understand this, it helps to separate the process into two distinct phases: **training** (fitting the model to the data) and **inference** (asking the model to predict a new point).

Splines and LOESS handle the mathematical heavy lifting in entirely different phases.

The Spline Approach: Eager Learning (Solve Once, Use Forever)

A spline is considered an "eager learner." It does all of its difficult mathematical work upfront during the training phase so that it doesn't have to think hard later.

1. **The Upfront Work (Training):** When you fit a spline, the algorithm looks at the entire dataset, divides the x-axis into segments, and solves a massive system of equations. For every single segment, it calculates a specific, permanent set of polynomial coefficients—like the β_0, β_1, β_2, and β_3 in a cubic equation ($y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3$).

2. **The "Global Set":** Once the algorithm finishes fitting, it discards the original training data. It only keeps a small lookup table of these β coefficients and the locations of the boundaries (knots).

3. **Instantaneous Prediction:** If you ask the fitted spline model to predict the y-value for a new data point at $x = 42$, the computer performs two incredibly fast steps:

    - It checks which segment $x = 42$ falls into.

    - It plugs 42 into that specific segment's pre-saved equation: $y = \beta_0 + \beta_1(42) + \beta_2(42^2) + \beta_3(42^3)$.

This takes just a few basic multiplications and additions. A modern CPU can calculate millions of spline predictions in a fraction of a second because it is just plugging a number into a known formula.

The LOESS Contrast: Lazy Learning (Solve Every Single Time)

LOESS is a "lazy learner." It does almost no math upfront. During the training phase, it essentially just memorizes the raw data.

Because it never calculates a global set of coefficients, it forces you to pay the computational tax during the prediction phase. If you ask LOESS to predict a value for $x = 42$, it must:

1. Scan the *entire* historical dataset to find the data points closest to 42.

2. Calculate custom mathematical weights for all of those neighboring points based on their exact distance from 42.

3. Run a brand new, computationally heavy weighted linear regression (which requires complex matrix algebra) just to find that one specific y-value.

4. Throw that localized regression model away.

If you want to predict a second point at $x = 43$, LOESS has to start over and do all of that heavy math completely from scratch.

Why Fitting An Exponential Curve Directly To Raw Data Often Produces A Distorted Model

Why fitting an exponential curve directly to raw data often produces a distorted model, and why log-transforming the data first yields a much more accurate result.

The difference between how statistical algorithms handle **absolute error** versus **relative error**.

The "Sum of Squares" Problem

When statistical software fits a curve to data, it typically uses a method called "Ordinary Least Squares." The algorithm's only goal is to minimize the total sum of the squared distances (deviations) between the actual data points and the fitted line.

In an exponentially growing dataset, this creates a massive imbalance:

- **Large values dominate:** If a massive data point at the end of the time series is off by 1,000, its squared penalty is 1,000,000.
- **Small values are ignored:** If an early data point is off by 5, its squared penalty is only 25.
- **The result:** To minimize that total sum, the algorithm aggressively bends the curve to perfectly align with the massive numbers at the top of the chart. In doing so, it happily sacrifices the accuracy of the smaller numbers at the bottom, causing the fitted line to systematically overshoot or undershoot them.

The Log-Transform Solution

The author advises taking the logarithm of the y-axis values before fitting a straight line to the data.

- **Equalizing the weights:** A logarithmic transformation compresses the scale so that it represents multiplicative changes rather than additive ones. The physical distance between 10 and 100 becomes identical to the distance between 100 and 1,000.
- **Proportional accuracy:** When you fit a standard linear regression to this log-transformed data, the algorithm is no longer blinded by massive absolute numbers. It gives equal mathematical weight to a 10% deviation on a small number and a 10% deviation on a large number.

Ultimately, for accurate data visualization, transforming the data first ensures the resulting trendline accurately reflects the underlying rate of change across the *entire* timeline, rather than just catering to the largest data points at the end.

Log-Linear, Log-Log and Linear-Log Plots

By selectively applying a logarithmic scale to the x-axis, the y-axis, or both, you can visually "straighten out" specific types of curved data.

Human eyes are excellent at detecting when data points deviate from a straight line, but very poor at judging the exact curvature of a parabola, exponential curve, or power-law curve. Transforming the axes mathematically forces these specific curves into straight lines, making analysis much easier.

Here is the mathematical mechanics behind the three core transformations:

1. Log-Linear Plots (For Exponential Growth)

- **Setup:** Linear x-axis, Logarithmic y-axis.
- **Target Function:** Exponential relationships ($y = ae^{bx}$).
- **The Math:** Taking the logarithm of both sides yields:

$$\log(y) = \log(a) + bx$$

- **Result:** This matches the standard equation for a straight line ($Y = mx + c$). By plotting $\log(y)$ against x, the growth rate (b) becomes the constant slope of the straight line.

2. Log-Log Plots (For Power Laws)

- **Setup:** Logarithmic x-axis, Logarithmic y-axis.
- **Target Function:** Power laws ($y = cx^{\alpha}$), where the variable is the base and the exponent is constant (e.g., $y = x^2$ or $y = x^{-3}$). This is common in physics and network theory.
- **The Math:** Taking the logarithm of both sides yields:

$$\log(y) = \log(c) + \alpha \log(x)$$

- **Result:** By plotting $\log(y)$ against $\log(x)$, the exponent (α) becomes the slope of the straight line. A steep straight line means a high exponent; a downward straight line means a negative exponent.

3. Linear-Log Plots (For Logarithmic Relationships)

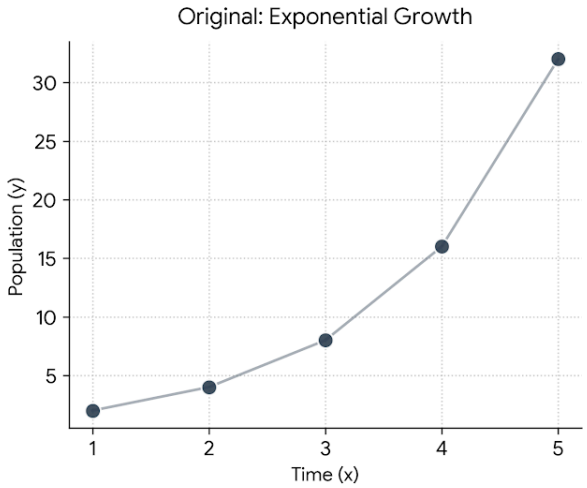
- **Setup:** Logarithmic x-axis, Linear y-axis.

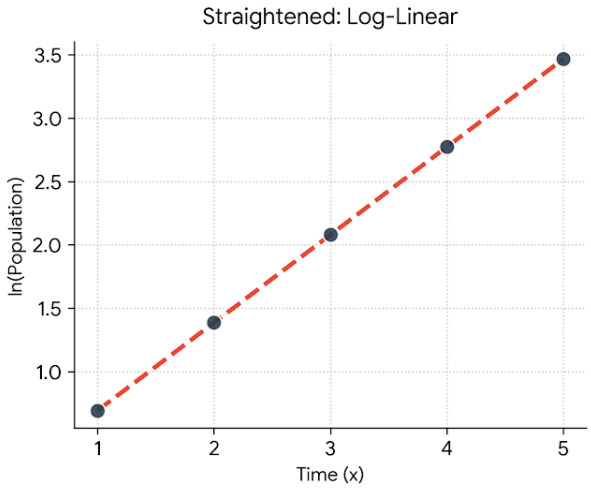
- **Target Function:** Logarithmic curves ($y = a \log(x) + b$). This describes a relationship that increases quickly at first but then flattens out over time.

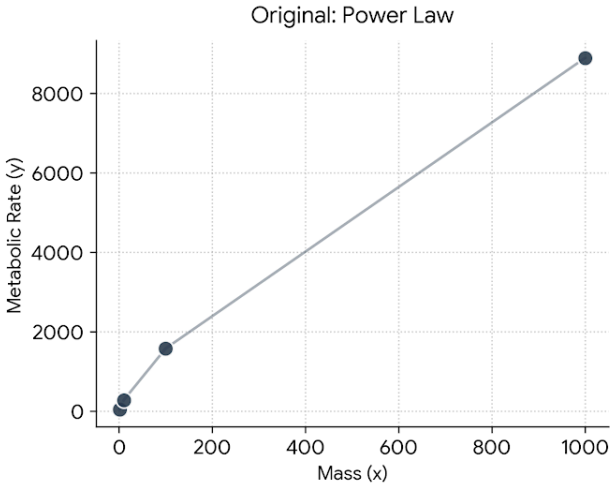
- **The Math:** The equation is already in a linear format if you treat $\log(x)$ as a single unit.

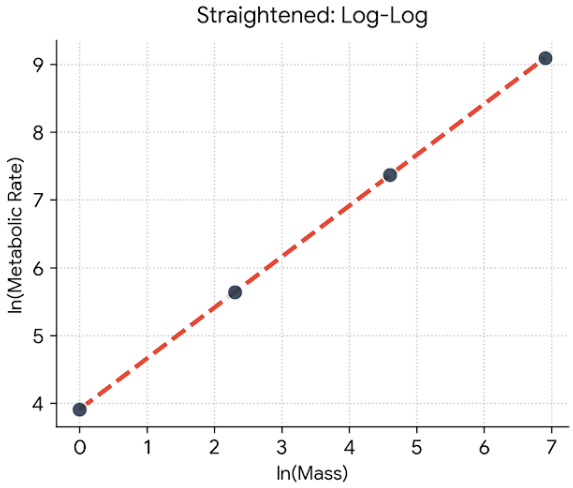
- **Result:** By visually compressing the x-axis logarithmically, the physical distance between 10 and 100 becomes the same as 100 and 1,000. This compression exactly cancels out the flattening of the curve, resulting in a straight line.

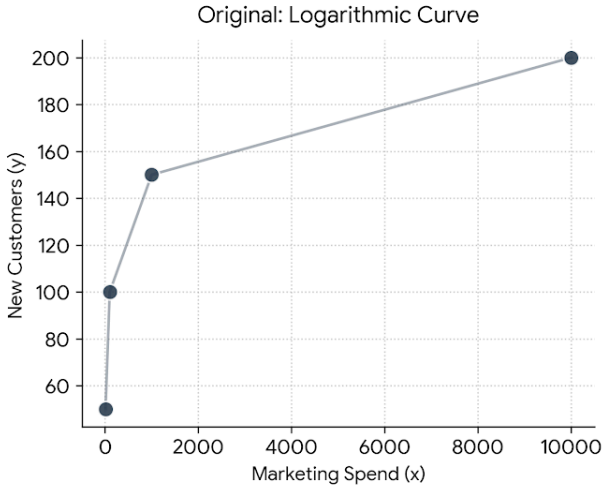
Examples of Data Straightening

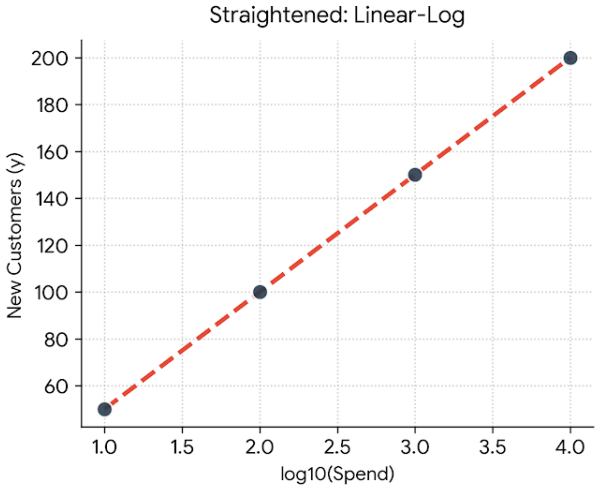












1. Log-Linear Plots (Straightening Exponential Growth)

Exponential curves accelerate upward rapidly. To straighten them, you compress only the y-axis using a logarithm, converting a constant multiplication rate into a constant addition rate (a straight line).

- **Real-World Scenario:** A bacteria population that doubles every hour.
- **Original Data (Curved):**
    - Time (x): 1, 2, 3, 4, 5
    - Population (y): 2, 4, 8, 16, 32
- **The Original Equation:** $y = 2^x$
- **The Transformation:** Take the natural logarithm of the y-axis values. The x-axis remains unchanged.
    - x: 1, 2, 3, 4, 5
    - $\ln(y)$: 0.69, 1.39, 2.08, 2.77, 3.47
- **The Linear Fit:** $Y = 0.693x + 0$. The resulting straight line has a slope of 0.693 (which is exactly $\ln(2)$), representing the continuous growth rate.

2. Log-Log Plots (Straightening Power Laws)

Power laws describe scale-invariant relationships where variables scale relative to each other. Because both the input and output are scaling dynamically, you must compress *both* axes to reveal the straight line.

- **Real-World Scenario:** Kleiber's Law, which states that an animal's metabolic rate scales to the $\frac{3}{4}$power of its body mass.
- **Original Data (Curved):**
    - Mass (x): 1, 10, 100, 1000
    - Metabolic Rate (y): 50, 281, 1581, 8891

- **The Original Equation:** $y = 50x^{0.75}$
- **The Transformation:** Take the natural logarithm of *both* the x-axis and the y-axis.
    - $\ln(x)$: 0, 2.30, 4.61, 6.91
    - $\ln(y)$: 3.91, 5.64, 7.37, 9.09
- **The Linear Fit:** $Y = 0.75X + 3.91$. The slope of the straightened line (0.75) exactly matches the power-law exponent.

3. Linear-Log Plots (Straightening Logarithmic Curves)

Logarithmic curves rise quickly at first but then violently flatten out. To straighten them, you compress the x-axis so that massive leaps in the independent variable are visually squashed into equal steps.

- **Real-World Scenario:** Diminishing returns in marketing. Spending your first $\$100$ gets you 50 customers, but to get the next 50 customers, you have to spend $\$1,000$.
- **Original Data (Curved):**
    - Spend (x): 10, 100, 1000, 10000
    - New Customers (y): 50, 100, 150, 200
- **The Original Equation:** $y = 50 \cdot \log_{10}(x)$
- **The Transformation:** Take the base-10 logarithm of the x-axis. The y-axis remains unchanged.
    - $\log_{10}(x)$: 1, 2, 3, 4
    - y: 50, 100, 150, 200
- **The Linear Fit:** $Y = 50X + 0$. The slope (50) tells you exactly how many new customers you acquire every time you multiply your marketing budget by 10.

When to Use Which Logarithm and How to Convert

The difference between ln and log is simply a difference in the underlying "base" they use to measure growth.

- **ln(x)** is the **natural logarithm**. It explicitly means the base is e (Euler's number, ≈ 2.718). It asks: *"To what power must I raise e to get x?"*

- **log(x)** usually means the **common logarithm**. In most middle school math classes, engineering contexts, and on hand-held calculators, it assumes a base of 10 ($\log_{10}$). It asks: *"To what power must I raise 10 to get x?"*

The "Programming Trap"

There is a massive trap here for data scientists and programmers. In standard mathematics, log implies base 10. But in almost all programming languages (Python, R, C++, Java), **the function log() defaults to base e**.
If you want base 10 in Python or R, you have to explicitly type log10().

When to use which?

- **Use log10 (Base 10) for human scales:** We count in base 10, so $\log_{10}$ perfectly counts the number of zeros (or "orders of magnitude"). A 7 on the Richter scale is exactly 10 times stronger than a 6. A pH of 3 is 10 times more acidic than a pH of 4.

- **Use ln (Base e) for calculus and modeling:** As established earlier, base e handles continuous rates of change perfectly. If you are running regressions, taking derivatives, or modeling compound growth, you should always default to ln.

- **Use $\log_2$ (Base 2) for computer science:** Used when analyzing binary trees, algorithm speeds (like Big O notation $O(\log n)$), or doubling times.

How to Convert Between Bases

Because all exponential and logarithmic growth is mathematically proportional, you can translate any base into any other base using a simple mathematical bridge.

1. Converting Logarithms (The Change-of-Base Formula)

If your calculator only has an ln button, but you need to calculate a base 10 logarithm (or a base 2 logarithm), you use this formula:

$$\log_{new}(x) = \frac{\log_{old}(x)}{\log_{old}(newbase)}$$

For example, to calculate $\log_{10}(50)$ using natural logs:

$$\log_{10}(50) = \frac{\ln(50)}{\ln(10)}$$

This tells us that the log10 curve is literally just the ln curve divided by a constant factor of $\ln(10)$ (which is about 2.303).

2. Converting Exponential Functions to Base e

If you have an equation modeled in base 10 or base 2, and you want to convert it into a natural base e equation for easier calculus, you use this rule:

Any base a can be rewritten as $e^{\ln(a)}$.

Therefore, to convert $y = a^x$ into the continuous format $y = e^{kx}$, you simply set $k = \ln(a)$.

Example: Converting $y = 10^x$ to base e

1. Find k: $\ln(10) \approx 2.303$
2. Rewrite: $y = e^{2.303x}$

These two equations are mathematically identical. The first says "it multiplies by 10 every time step." The second says "it grows continuously at a rate of 230.3% per time step." Both result in the exact same curve.